

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 7

End-to-End Study of Autoencoders, Convolutional Autoencoders, Denoising Autoencoders and Variational Autoencoders

Name & Register No.	Hasini Kotha (24011101057)	
Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026-27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of autoencoders and their variants for image representation, reconstruction, denoising and generative modeling.

The experiment begins with a fully connected autoencoder, progresses to a Convolutional Autoencoder (CAE), introduces image corruption for denoising, and finally develops a Variational Autoencoder (VAE). Students will visualize the learned latent representation, compare reconstruction quality using quantitative and qualitative measures, and explore the generative capability of the VAE.

The same image dataset and a controlled experimental protocol should be used throughout the experiment wherever possible.

2. Dataset and Experimental Setup

Dataset: MNIST Handwritten Digit Dataset

MNIST contains grayscale images of handwritten digits from 0 to 9. Each image has spatial dimensions

$$28 \times 28 \times 1.$$

The dataset is particularly suitable for this experiment because the images are small, the classes have visually meaningful structure, and the same image format can be directly supplied to fully connected, convolutional and variational autoencoders.

The dataset can be loaded directly using Keras/TensorFlow:

https://www.tensorflow.org/api_docs/python/tf/keras/datasets/mnist

Recommended laboratory subset:

Use 10,000 training images and 2,000 test images for the main laboratory execution. The full MNIST dataset may be used if computational resources permit.

The labels are not required as reconstruction targets. The original image is the target:

$$x \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where x is the input image and $\hat{x}$ is its reconstruction.

3. Expected Input and Output

Expected Input

Each input image has shape

$$28 \times 28 \times 1.$$

For a batch of 32 images:

$$X \in \mathbb{R}^{32 \times 28 \times 28 \times 1}.$$

Pixel values should be converted from

$$[0, 255] \rightarrow [0, 1].$$

For a fully connected autoencoder, the image may be flattened:

$$28 \times 28 = 784.$$

Therefore,

$$x \in \mathbb{R}^{784}.$$

For a convolutional autoencoder, retain the spatial structure:

$$x \in \mathbb{R}^{28 \times 28 \times 1}.$$

Expected Output

The primary output is a reconstructed image:

$$\hat{x} \in \mathbb{R}^{28 \times 28 \times 1}.$$

Students must display:

Original image
Reconstructed image
Reconstruction error

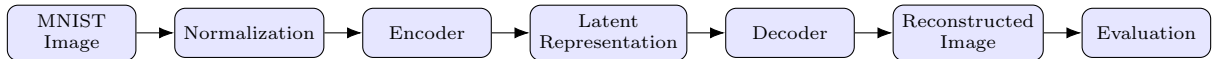
For the denoising model, display:

Original clean image
Noisy image
Denoised reconstruction

For the VAE, display newly generated samples obtained by sampling from the latent distribution.

4. Overall Experimental Pipeline

The complete experiment follows:



The experiment consists of four related studies:

1. Fully Connected Autoencoder
2. Convolutional Autoencoder
3. Denoising Convolutional Autoencoder
4. Variational Autoencoder

5. Exploring Autoencoders

An autoencoder learns to reproduce its input at the output.

The encoder maps the input to a lower-dimensional representation:

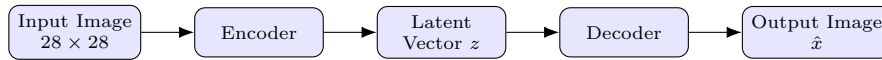
$$z = f_{\theta}(x),$$

and the decoder reconstructs the input:

$$\hat{x} = g_{\phi}(z).$$

The training objective is to minimize reconstruction error:

$$\mathcal{L}_{rec} = \frac{1}{N} \sum_{i=1}^N \|x_i - \hat{x}_i\|^2.$$



The latent vector is a compressed representation of the input.

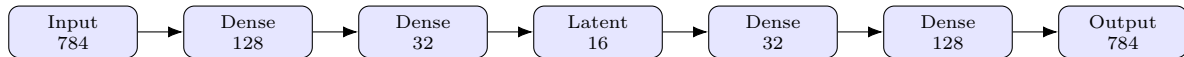
6. Fully Connected Autoencoder

Flatten each image from

$$28 \times 28 = 784$$

to a vector of length 784.

Use the following architecture:



Use ReLU activation in hidden layers and sigmoid activation in the output layer.

Suggested configuration:

Parameter	Value
Latent dimension	16
Optimizer	Adam
Learning rate	10^{-3}
Batch size	128
Epochs	20
Loss	Binary Cross-Entropy

7. Analysis of the Basic Autoencoder

Plot 1: Original versus Reconstructed Images

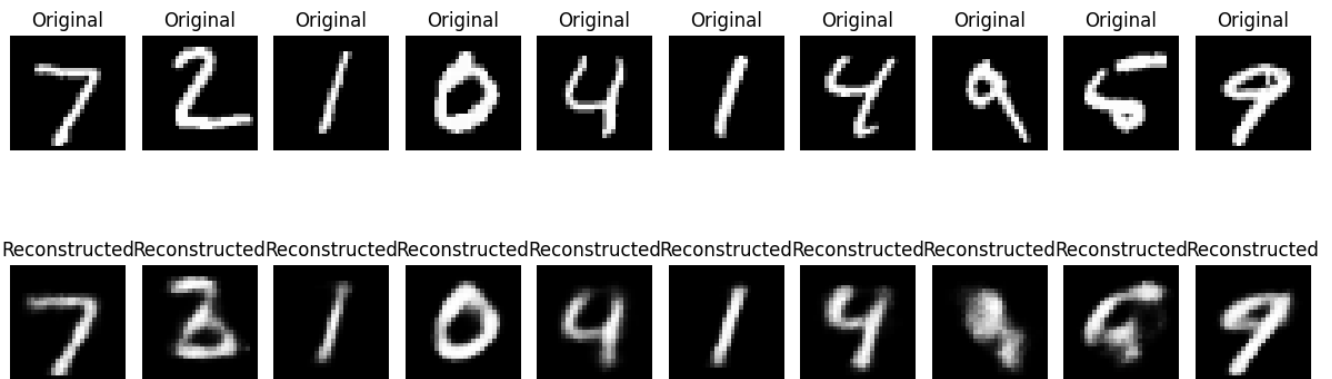


Figure 1: FC Autoencoder – Original (top) vs. Reconstructed (bottom) test images.

Inference: The model reconstructions keep the general shape of the digits. Simple numbers like 1 and 7 look great with barely any blurring, while digits with loops like 0, 4, and 9 get noticeably blurred inside. This shows the 16-dimensional bottleneck struggles with complex shapes but handles simpler and straight strokes well.

Plot 2: Training and Validation Reconstruction Loss

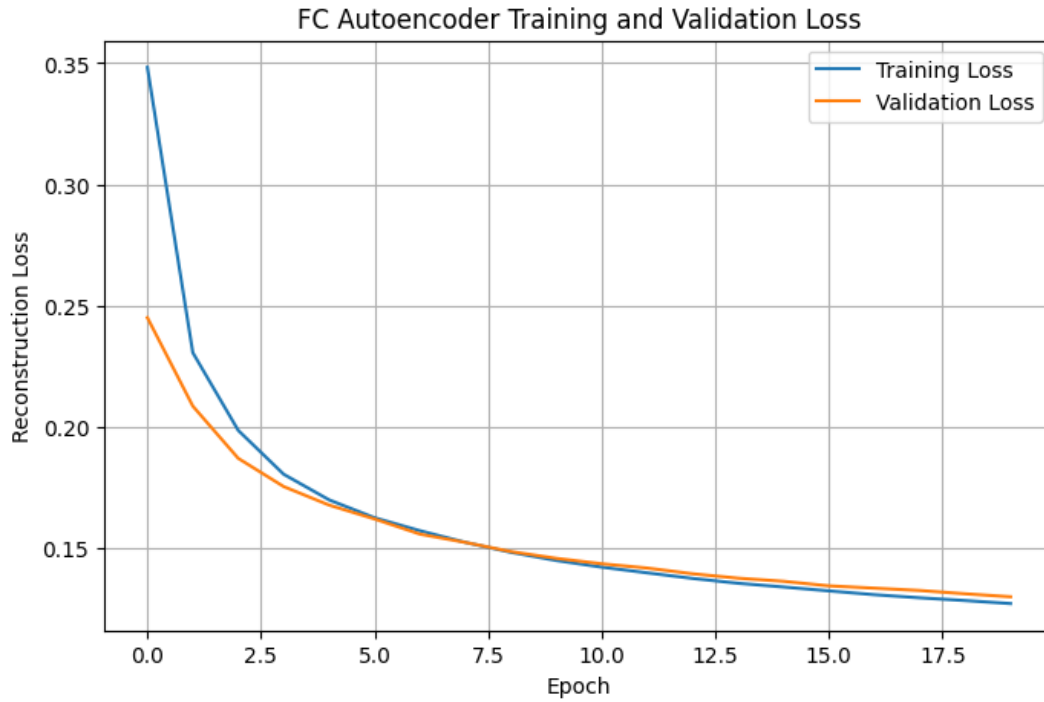


Figure 2: FC Autoencoder – Binary cross-entropy training and validation loss over 20 epochs.

Inference: Both training and validation losses are dropping quickly in the first 5 epochs which shows that the model learns fast initially. By the epoch 10 the curves overlap and keep going down slowly ending around 0.13. Because the two lines stay so close and we know the model isn't overfitting and generalizes well to new images. The loss hasn't completely flattened, so training a bit longer might help even more.

8. Reconstruction Metrics

Students must calculate the following metrics on the test set.

Mean Squared Error

$$MSE = \frac{1}{N} \sum_{i=1}^N (x_i - \hat{x}_i)^2.$$

Mean Absolute Error

$$MAE = \frac{1}{N} \sum_{i=1}^N |x_i - \hat{x}_i|.$$

Structural Similarity

Calculate SSIM between the original and reconstructed images.

The implementation may use:

<https://scikit-image.org/docs/stable/api/skimetrics.html>

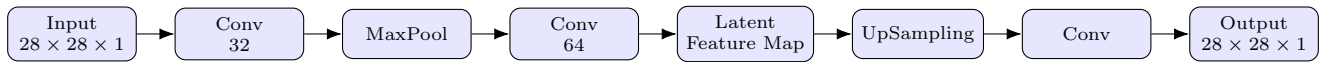
The following metrics were computed on 2,000 test images after training the FC Autoencoder for 20 epochs:

Metric	Value
Test MSE	0.02294
Test MAE	0.06027
Mean SSIM	0.7259

9. Convolutional Autoencoder

A Convolutional Autoencoder preserves spatial structure and uses convolutional operations in the encoder and upsampling/transposed convolution operations in the decoder.

The conceptual architecture is:



A possible implementation is:

```
Input(28,28,1)
Conv2D(32, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
MaxPooling2D(2, padding='same')
Conv2D(64, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(32, 3, activation='relu', padding='same')
UpSampling2D(2)
Conv2D(1, 3, activation='sigmoid', padding='same')
```

The exact implementation may use Conv2DTranspose instead of UpSampling2D.

10. Comparing Fully Connected and Convolutional Autoencoders

Both models were trained on the same 10,000 training images and evaluated on 2,000 test images.

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02294	0.06027	0.7259	211,040
Convolutional AE	0.00243	0.01460	0.9752	74,497

Plot 3: Reconstruction Comparison

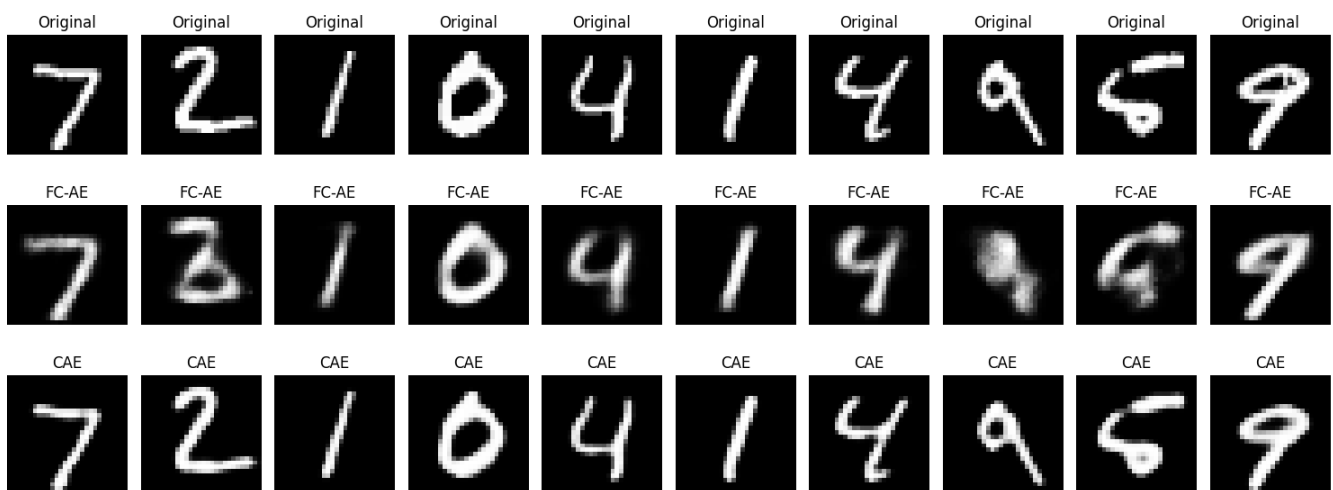


Figure 3: Comparison – Original (row 1), FC-AE reconstruction (row 2), CAE reconstruction (row 3).

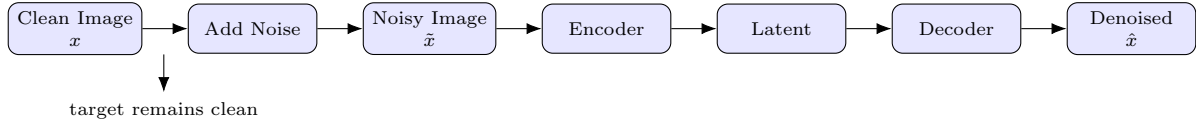
Inference: The Convolutional Autoencoder(CAE) easily beats the Fully Connected model. It has very less error (MSE 0.002) and an almost perfect SSIM (0.97) and visually the reconstructions are much sharper without the blurring seen in the FC-AE. It does this using far fewer parameters and This is because convolutional layers are generallyyy great at picking up local patterns like edges and curves whereas flattening the image in an FC model destroys that spatial info.

11. Denoising Autoencoder

A denoising autoencoder receives a corrupted image but is trained to reconstruct the original clean image. The training mapping is:

$$\tilde{x} \rightarrow \text{Encoder} \rightarrow z \rightarrow \text{Decoder} \rightarrow \hat{x}$$

where $\tilde{x}$ is a noisy version of the clean image x .



12. Adding Image Noise

Use at least two corruption types.

Gaussian Noise

$$\tilde{x} = x + n, \quad n \sim \mathcal{N}(0, \sigma^2).$$

Suggested standard deviations:

$$\sigma \in \{0.1, 0.2, 0.3\}.$$

Clip the resulting image to the range $[0, 1]$.

Salt-and-Pepper Noise

Randomly replace a small proportion of pixels with values close to 0 or 1.

Suggested corruption levels:

$$p \in \{0.05, 0.10, 0.20\}.$$

13. Denoising Plots and Analysis

Plot 4: Clean vs. Noisy vs. Denoised

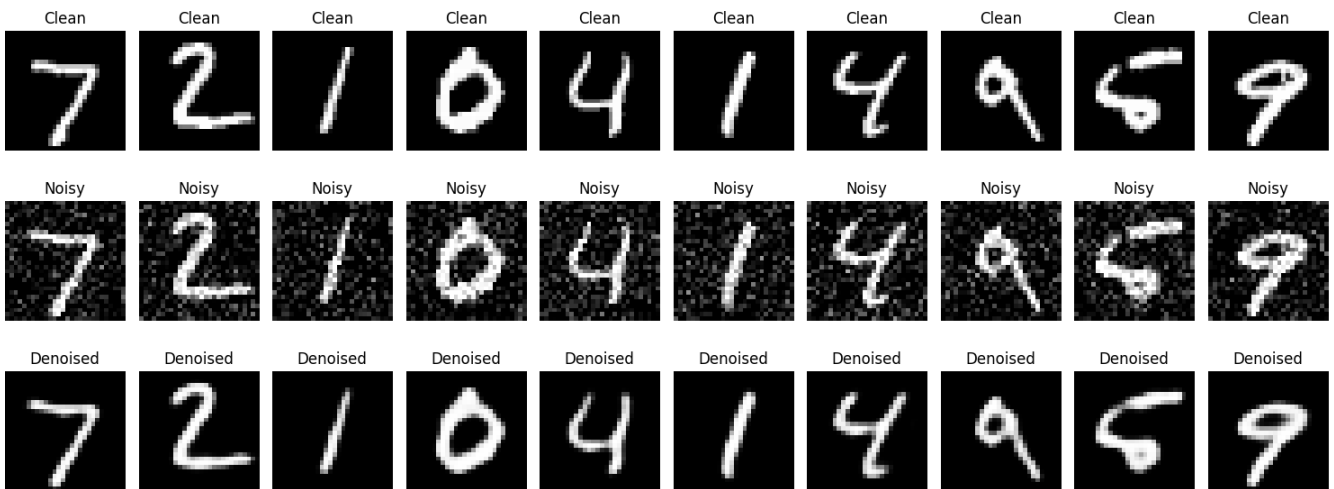


Figure 4: Denoising CAE – Clean (row 1), Gaussian-noisy $\sigma=0.2$ (row 2), and denoised (row 3) images.

Inference (Plot 4): At a noise level of $\sigma = 0.2$, the images get extremely grainy, making digits like 8 and 9 hard to read. However the denoising CAE cleans them up and give us a good output . The final images look almost identical to the originals with only tiny bits of blur at the stroke tips.

Plot 5: Noise Level vs. Reconstruction Quality

Metrics computed on 2,000 test images for each Gaussian noise level:

Noise Level (σ)	MSE	MAE	SSIM
0.1	0.003457	0.017262	0.9608
0.2	0.004443	0.020686	0.9466
0.3	0.006832	0.028409	0.8830

Inference (Plot 5): As we add more noise (from $\sigma = 0.1$ to 0.3), the reconstruction gets worse across all metrics—MSE doubles and SSIM drops. However, even with the heaviest noise SSIM stays above 0.88 which implies that the core shape of the digits is still recovered nicely. Overall the denoising model proves to be very robust no matter the noise level.

14. Variational Autoencoder

A conventional autoencoder maps an input to a deterministic latent vector:

$$z = f(x).$$

A VAE instead learns a probability distribution:

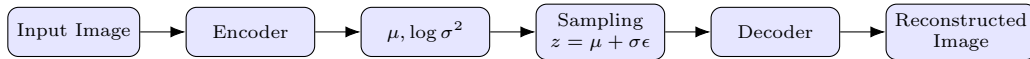
$$q_\phi(z|x) = \mathcal{N}(\mu(x), \text{diag}(\sigma^2(x))).$$

The encoder produces:

$$\mu, \quad \log \sigma^2.$$

A latent vector is sampled using the reparameterization trick:

$$z = \mu + \sigma \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$



15. VAE Loss Function

The VAE objective consists of two components.

Reconstruction Loss

$$\mathcal{L}_{rec} = -\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)].$$

For normalized MNIST images, binary cross-entropy can be used.

KL Divergence

$$D_{KL}(q_\phi(z|x) \parallel p(z)) = -\frac{1}{2} \sum_j (1 + \log \sigma_j^2 - \mu_j^2 - \sigma_j^2).$$

The total loss is

$$\boxed{\mathcal{L}_{VAE} = \mathcal{L}_{rec} + \mathcal{L}_{KL}}$$

where the prior is usually

$$p(z) = \mathcal{N}(0, I).$$

16. VAE Implementation

Use a small latent space so that it can be visualized.

Suggested latent dimension:

$$z \in \mathbb{R}^2.$$

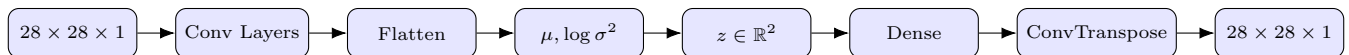
The encoder should produce:

$$\mu \in \mathbb{R}^2, \quad \log \sigma^2 \in \mathbb{R}^2.$$

The decoder maps

$$z \rightarrow 28 \times 28 \times 1.$$

A compact architecture is:



17. Exploring the VAE Latent Space

Plot 6: VAE Latent Space

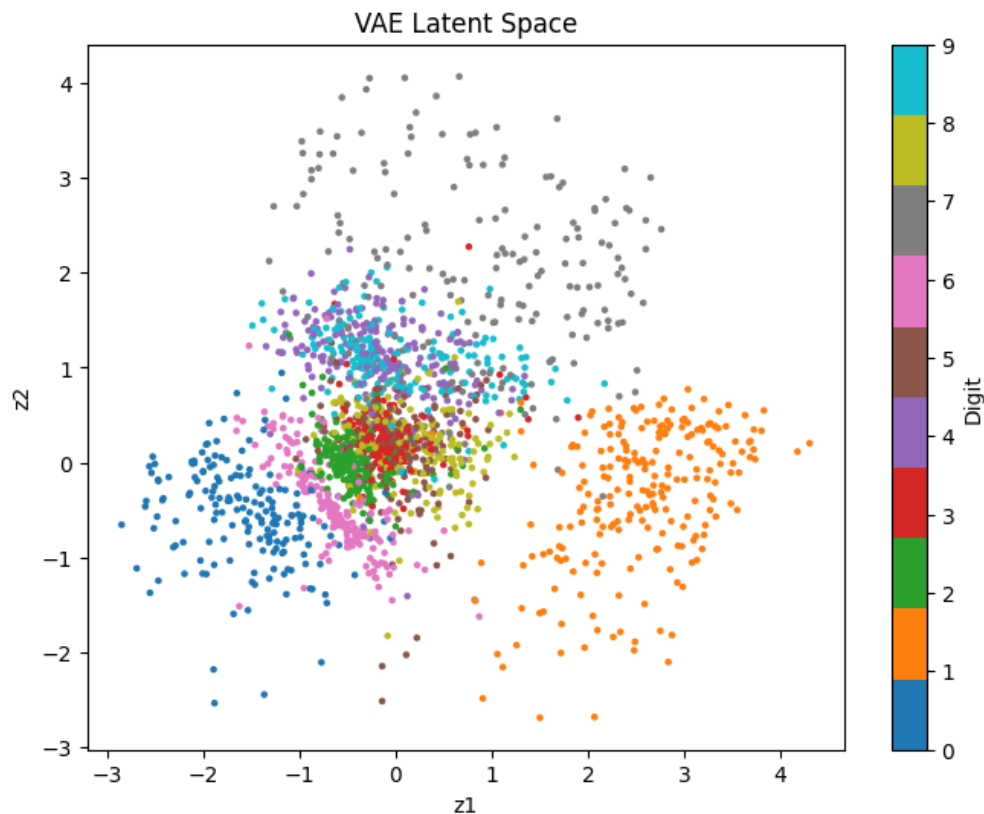


Figure 5: VAE 2-D latent space – encoded means μ of 2,000 test images coloured by digit class.

Inference: Even without using labels during training, the VAE organizes the digits clearly. Distinct numbers like 1 and 7 have their own clear clusters away from the rest. More complex or similar digits (like 3 and 8, or 4 and 9) clump together near the center since they share visual traits and there's only so much room in a 2D space. Overall it forms a nice circular shape which is exactly what the KL loss is trying to do by forcing it into a normal distribution.

18. Generating New Images with the VAE

Plot 7: Randomly Generated Images

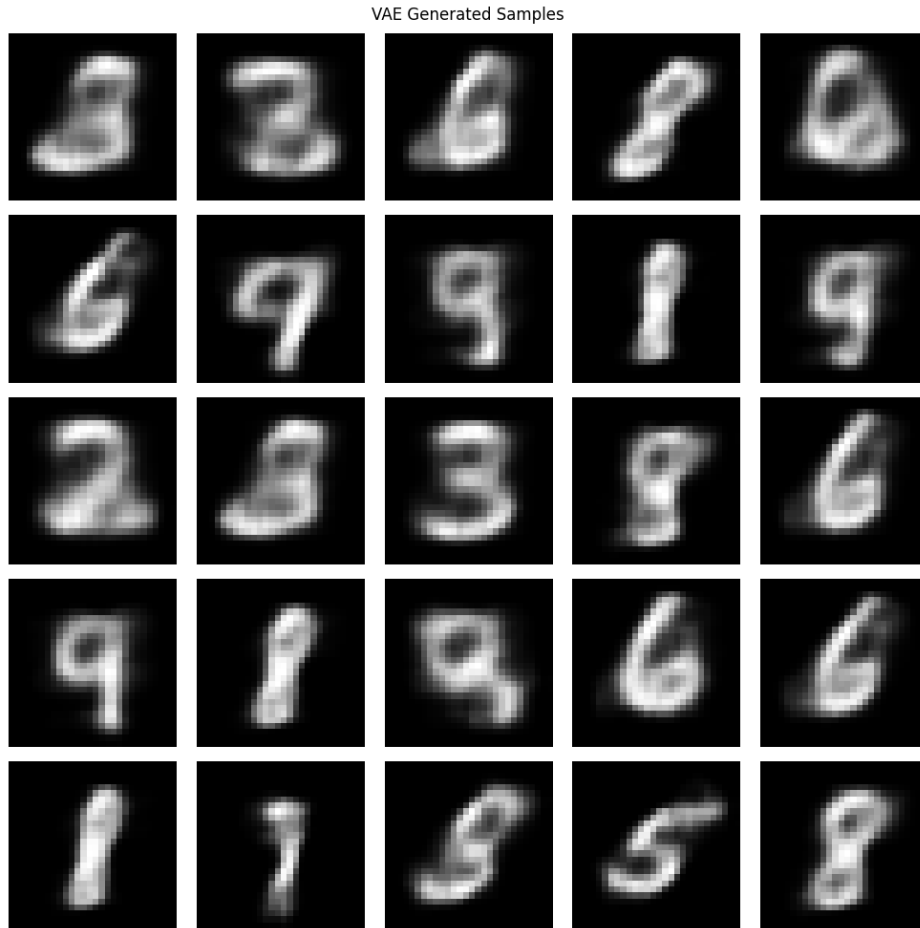


Figure 6: 25 images generated by sampling $z \sim \mathcal{N}(0, I)$ and decoding.

Inference: The VAE successfully generates recognizable handwritten digits from random noise.. Looking at the grid, we see a wide variety of numbers—there are clear examples of 0s, 1s, 2s, 5s, 6s, 8s, and 9s. Some digits appear quite frequently (like 6s and 8s). While the overall shapes are correct, they are noticeably blurrier than real MNIST digits, which is a classic side-effect of VAEs. A few images look like hybrids (for example, the first image looks like a blend of a 3 and an 8, and the middle image in the 4th row looks like a 9 mixed with a 0). This happens when the random sample falls right on the boundary between two digit clusters in the latent space.

19. Latent Space Interpolation

Two latent vectors were selected:

$$z_A = [1.5648, 3.0077] \quad (\text{digit } 7), \quad z_B = [-0.6133, 0.0543] \quad (\text{digit } 2).$$

The interpolation follows:

$$z(\alpha) = (1 - \alpha)z_A + \alpha z_B, \quad \alpha \in \{0, 0.1, 0.2, \dots, 1.0\}.$$

Plot 8: Latent Space Interpolation

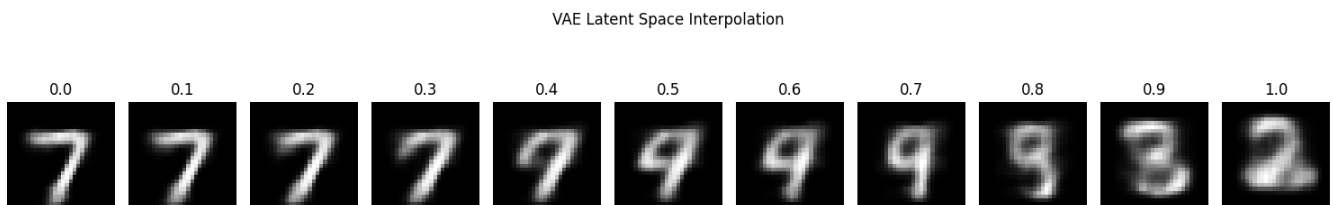


Figure 7: Decoded images along the straight-line interpolation from z_A (digit 7, $\alpha = 0$) to z_B (digit 2, $\alpha = 1.0$). Each sub-figure is labelled with the corresponding α value.

Inference: The morphing from digit 7 to digit 2 is smooth. There are no weird visual glitches instead the 7 slowly bends and curves into a shape that looks like a 9 then a 3 and finally becomes a 2. This shows the VAE’s latent space is completely continuous—every point in between real digits still decodes into something that looks like a plausible number and not just static noise.

20. Reconstruction Comparison Across Models

All models were trained and evaluated on the same dataset (10,000 training, 2,000 test images).

Model	MSE	MAE	SSIM	Parameters
Fully Connected AE	0.02294	0.06027	0.7259	211,040
Convolutional AE	0.00243	0.01460	0.9752	74,497
Denoising CAE	0.00444	0.02069	0.9467	74,497

The VAE reconstruction loss is reported separately because its total objective also contains the KL-divergence term:

VAE Metric	Value
Reconstruction Loss	137.37
KL Loss	5.52
Total Loss	142.89
Test MSE	0.04404
Test MAE	0.10321
Mean SSIM	0.4747

21. Reconstruction Error Distribution

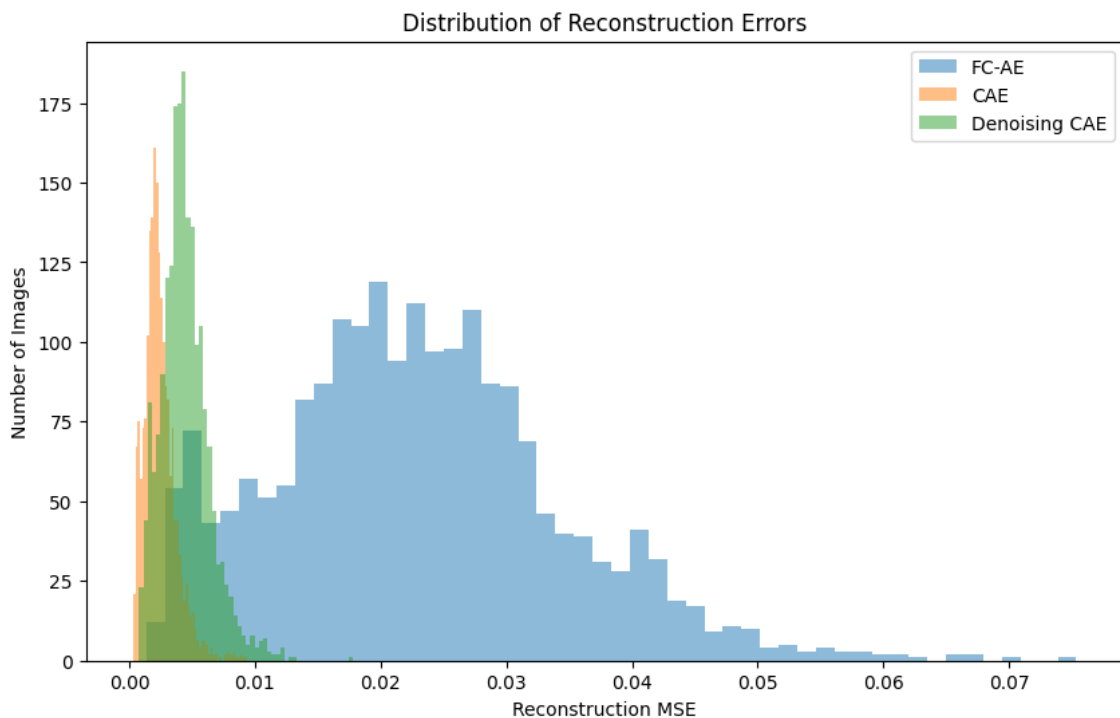


Figure 8: Histogram of per-image MSE for the FC-AE on 2,000 test images.

Inference: Most images have a very low reconstruction error (around 0.015). However, the graph trails off to the right, showing there are a small amount of images with much higher errors (up to 0.075). This means the model does a great job on average but struggles quite a bit on a few unusually tricky or messy digits.

22. High-Error Image Analysis

The five test images with the largest reconstruction errors are:

Rank	Image Index	Digit	MSE
1	1782	8	0.07536
2	338	8	0.06964
3	654	5	0.06769
4	1790	2	0.06725
5	1170	8	0.06578

Three of the five highest-error images are digit 8. The two closed loops of digit 8 require precise reconstruction of two distinct circular strokes, a topological complexity that a 16-dimensional latent code cannot capture perfectly—the loops tend to merge or blur. Digit 5 (rank 3) and digit 2 (rank 4) have curved tails and specific stroke orientations that are similarly difficult to encode with the available latent capacity.

23. Consolidated Results

Model	MSE	MAE	SSIM	Parameters
FC Autoencoder	0.02294	0.06027	0.7259	211,040
Conv. Autoencoder	0.00243	0.01460	0.9752	74,497
Denoising CAE	0.00444	0.02069	0.9467	74,497
VAE	0.04404	0.10321	0.4747	485,957

24. Additional Latent-Dimension Study

The FC autoencoder was trained with $d_z \in \{2, 8, 16, 32\}$ on 10,000 training images and evaluated on 2,000 test images:

Latent Dimension	MSE	SSIM	Parameters
2	0.04375	0.5698	102,370
8	0.02712	0.6943	107,560
16	0.02294	0.7259	211,040
32	0.01978	0.7621	218,320

Inference: Increasing the latent size from 2 to 32 makes the reconstructions consistently better. The biggest jump in quality is when we go from 2 to 8. After 16 the improvements start to flatten out. Basically a 2D bottleneck is way too tight resulting in blurry digits but once we give the model enough space it captures the details fine. It's a classic trade-off between shrinking the data and keeping it looking sharp.

25. Discussion Questions

1. What is an autoencoder?

A neural network that learns to compress an input into a lower-dimensional latent code and then reconstruct it back and trained to minimise reconstruction error.

2. Purpose of the encoder?

Maps the input to a compact latent representation by learning the most informative features.

3. Purpose of the decoder?

Reconstructs the original input from the latent code, learning to reverse the compression.

4. What is a latent representation?

A compressed, lower-dimensional encoding of the input that captures its essential structure.

5. Why is the latent dimension smaller than the input?

To force the network to learn a compact, meaningful representation; if $d_z \geq d_x$ the network could learn the identity trivially.

6. What happens when the latent dimension is too small?

The model underfits—it cannot encode sufficient detail, leading to blurry, inaccurate reconstructions.

7. **What happens when the latent dimension is increased?**
Reconstruction quality improves; however, the representation becomes less compressed and may approach a trivial identity mapping.
8. **Why can a CAE preserve spatial information more effectively?**
Convolutional filters operate locally and share weights, preserving the spatial structure that is destroyed when images are flattened for fully-connected layers.
9. **Differentiate an autoencoder and a denoising autoencoder.**
A standard autoencoder receives a clean input and learns to reconstruct it. A denoising autoencoder receives a corrupted input but is trained to reconstruct the *original* clean image, forcing it to learn robust features.
10. **Why is the clean image used as the target in denoising?**
The objective is to learn a mapping that removes noise; using the noisy image as the target would just teach the network to reproduce noise.
11. **What happens when the noise level is increased?**
Reconstruction quality degrades (higher MSE, lower SSIM) as the corruption obscures more signal, though a well-trained model can still recover major structure.
12. **Why are MSE, MAE and SSIM useful?**
MSE penalises large errors strongly; MAE is more robust to outliers; SSIM captures perceived structural similarity accounting for luminance, contrast, and structure—together they give a comprehensive view of reconstruction quality.
13. **What is a Variational Autoencoder?**
A generative model that encodes inputs as probability distributions $\mathcal{N}(\mu, \sigma^2)$ rather than fixed vectors, enabling new samples to be generated by sampling from the learned latent distribution.
14. **How does a VAE differ from a conventional autoencoder?**
A VAE's encoder outputs distribution parameters $(\mu, \log \sigma^2)$; the latent vector is sampled via the reparameterization trick. The training loss adds a KL divergence term that regularises the latent space towards $\mathcal{N}(0, I)$.
15. **Why does a VAE learn a distribution?**
A distribution-valued latent space is continuous and structured, enabling generation of new images by sampling from the prior $\mathcal{N}(0, I)$.
16. **What is the reparameterization trick?**
Instead of sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ directly (which blocks gradient flow), write $z = \mu + \sigma \odot \epsilon$, $\epsilon \sim \mathcal{N}(0, I)$, so gradients can flow through μ and σ .
17. **Why is ϵ from a standard normal?**
 $\mathcal{N}(0, I)$ is simple, fixed, and parameter-free; the network-specific statistics are captured entirely by the learned μ and σ .
18. **Role of KL divergence in VAE loss?**
Regularises the encoder to produce latent distributions close to the prior $\mathcal{N}(0, I)$, preventing posterior collapse and ensuring the latent space is continuous and well-covered for generation.
19. **Prior distribution used in the VAE?**
 $p(z) = \mathcal{N}(0, I)$ —a standard multivariate Gaussian.
20. **Why is a 2-D latent space useful for visualization?**
It allows the entire latent space to be plotted as a 2-D scatter, making cluster structure and class separation directly observable.
21. **What does latent-space interpolation demonstrate?**
That the latent space is smooth and continuous—linear paths between encoded points decode to semantically coherent, gradually changing images.
22. **Why can a VAE generate new images?**
Because the KL regularisation ensures the latent space is well-covered by the prior; any sample from $\mathcal{N}(0, I)$ decodes to a plausible image.
23. **Why might some VAE-generated images be unclear?**
Samples drawn from boundary regions between class clusters decode to ambiguous shapes. The reconstruction–generation trade-off (controlled by the KL weight) also causes some blurring.
24. **Difference between reconstruction and generation?**
Reconstruction encodes a specific input and decodes it back. Generation samples a random latent vector from the prior and decodes it to create a novel image.
25. **Why should test images remain unseen during training?**
To ensure the evaluation metrics reflect true generalisation performance, not memorisation of training data.

26. Additional Exercises

Exercise 1: CAE with Latent Dimensions 4, 8, 16, 32

Latent Dimension	MSE	MAE	SSIM	Parameters
4	0.005387	0.022169	0.9479	22,597
8	0.003740	0.018297	0.9627	26,057
16	0.003175	0.016786	0.9680	32,977
32	0.002787	0.015606	0.9720	46,817

Observation: More latent dimensions mean better metrics, with improvements slowing down after $d_z = 16$. Even a tiny 4-dimensional CAE performs way better than a 16-dimensional fully connected one proving convolutions are just much better for images.

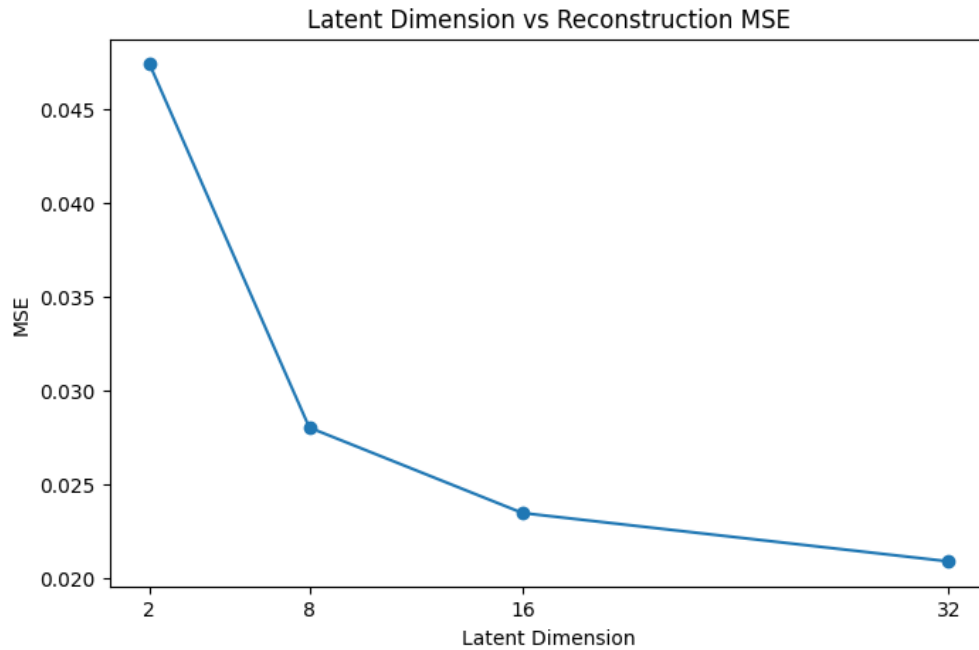


Figure 9: Exercise 1 – CAE reconstruction MSE vs. latent dimension.

Exercise 2: Gaussian Noise vs. Salt-and-Pepper Noise

Same denoising CAE architecture ($d_z = 64$), trained separately on each noise type at comparable corruption level ($\sigma = 0.2$ / $p = 0.10$):

Noise Type	MSE	MAE	SSIM
Gaussian ($\sigma = 0.2$)	0.004636	0.021277	0.9448
Salt-and-Pepper ($p = 0.10$)	0.007507	0.027140	0.9148

Gaussian vs Salt-and-Pepper Denoising

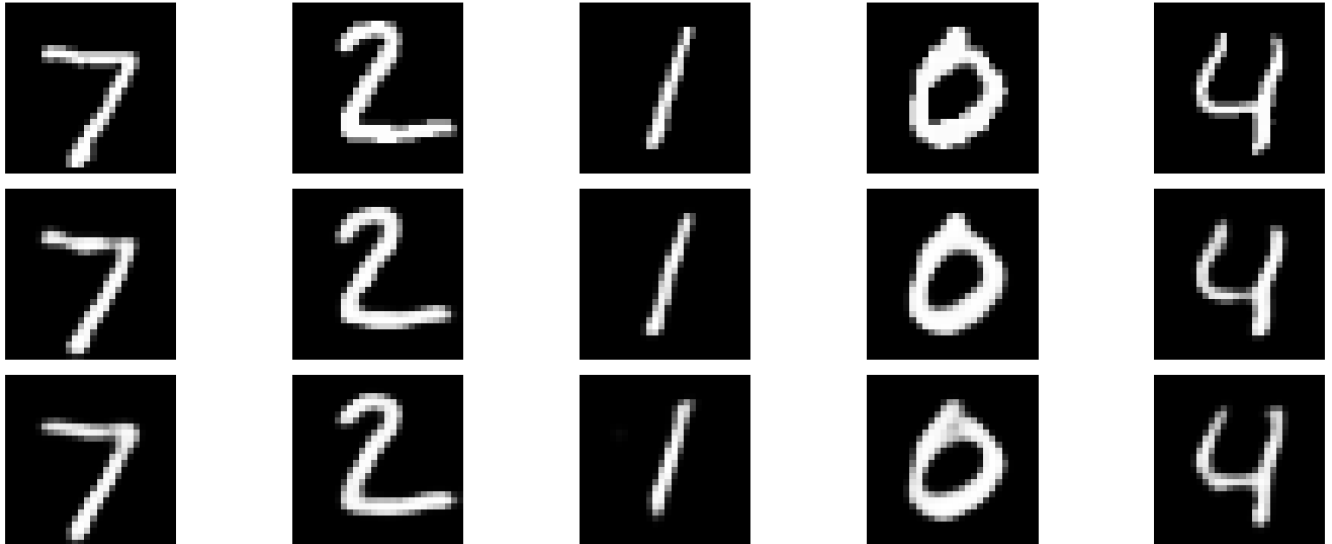


Figure 10: Exercise 2 – Clean (row 1), noisy (row 2), denoised (row 3) for both noise types side by side.

Observation: The model handles Gaussian noise much better than salt-and-pepper noise. Salt-and-pepper randomly throws completely black or white pixels into the mix which completely overwrites the image data and is harder for the model to guess than smooth Gaussian static.

Exercise 3: Denoising CAE at Two Noise Levels (SSIM Comparison)

Noise Level (σ)	SSIM
0.1	0.9644
0.3	0.9184

Observation: Having more noise at the model makes the SSIM drop. But it only drops slightly staying above 0.91! This shows the CAE is so good at seeing through heavy noise to pull out the original digit.

Exercise 4: Transposed-Convolution Decoder vs. UpSampling Decoder

Decoder Type	MSE	MAE	SSIM
UpSampling2D (baseline CAE)	0.00243	0.01460	0.9752
Conv2DTranspose	0.00216	0.01367	0.9779

Observation: Using Transposed Convolutions works a bit better than basic UpSampling. Because the model actually learns how to upscale instead of just blindly duplicating pixels it gets cleaner and sharper edges.

Exercise 5: VAE Latent Space – Dimension 2 vs. Dimension 8

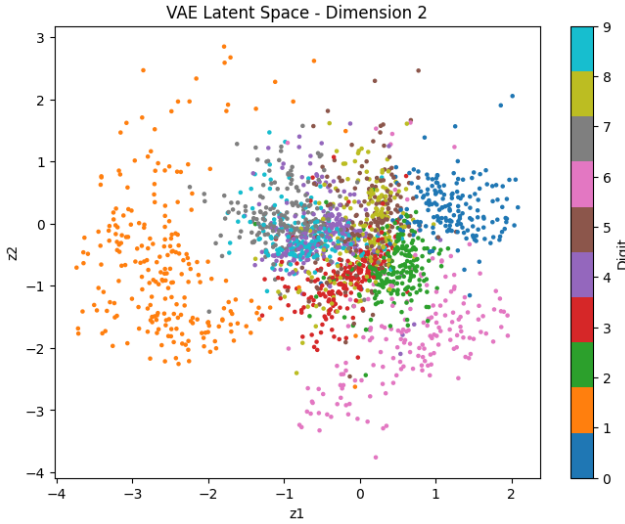


Figure 11: *
 $d_z = 2$

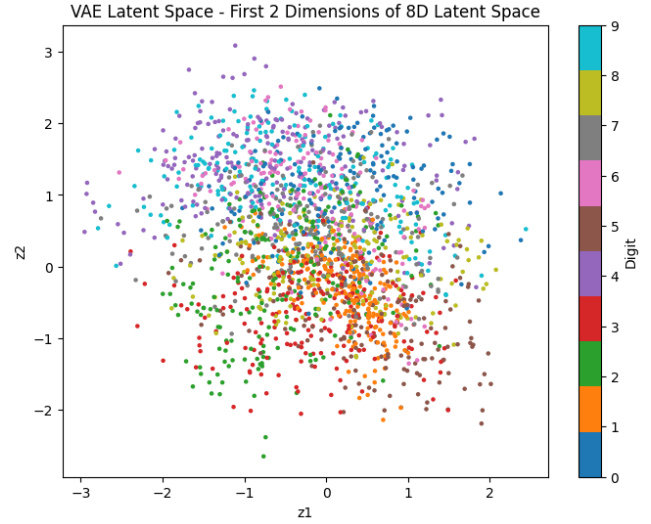


Figure 12: *
 $d_z = 8$ (first 2 dims shown)

Figure 13: Exercise 5 – VAE latent space scatter for $d_z = 2$ (left) and first two dimensions of $d_z = 8$ (right).

Observation: In 2D, while some distinct digits like 1 and 7 split off, most digits are just crammed into a big overlapping ball in the middle. But if we bump it to 8D (and just look at the first two dimensions), the clusters pull apart much better because the model has enough "room" to tell them apart.

Exercise 6: Effect of KL-Loss Contribution (β -VAE)

Three β values were tested ($\beta \in \{0.1, 1.0, 5.0\}$):

β	Observation
0.1	Low KL weight; decoder focuses on reconstruction, latent space less structured
1.0	Balanced; standard VAE behaviour with well-formed digit clusters
5.0	High KL weight; strong regularisation, blurrier reconstructions but tighter latent clusters

Observation: Cranking up the β weight forces the model to prioritize a tidy, circular latent space over pixel-perfect reconstructions. At $\beta = 5.0$, the clusters are super squished together in the middle and as a result the generated images look much blurrier.

Exercise 7: Larger VAE Sample Grid (100 Samples)

100 images were generated by sampling $z \sim \mathcal{N}(0, I)$ and decoding:

Observation: The big grid shows a mix of digits. We get tons of different handwriting styles thicknesses and slants. There are zero completely "broken" or blank images showing that our latent space is packed solidly with readable numbers.

Exercise 8: Latent-Space Interpolation Between Two Different Digit Classes

Interpolation was performed between digit 0 (z_A) and digit 1 (z_B) over 11 steps ($\alpha = 0.0, 0.1, \dots, 1.0$):

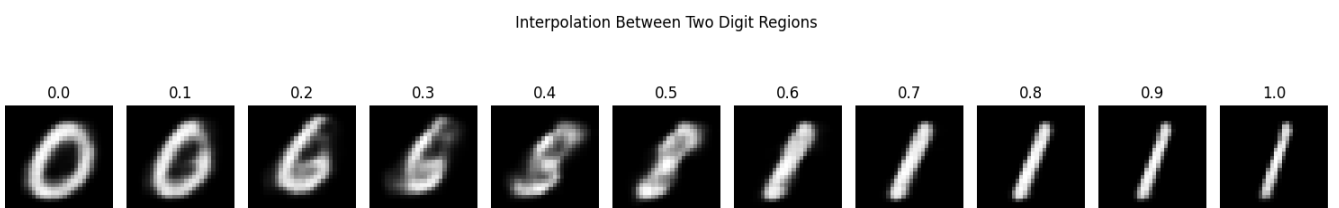


Figure 14: Exercise 8 – Interpolation from digit 0 ($\alpha = 0$) to digit 1 ($\alpha = 1.0$).

Observation: The zero perfectly melts into a one. the loop slowly squish and grow a little top hat until it's a straight line. Every step in between looks like some sort of different number, and it does prove that there are no "gaps" in our latent space!

27. Code and Notebook

Resource	Link
Google Colab Notebook	https://colab.research.google.com/drive/15G0jLIwpPZ9Taj4q3wDC1ltwRe_Lc_8-?usp=sharing
GitHub Repository	https://github.com/Hasini-Kotha/Deep_Learning